

Reviewer Pack — Cycle Matroid Rank Inverse Proportional to Resolution Proof ...

Entry 1661f91ede76 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 22:27:03 UTC

Cycle Matroid Rank Inverse Proportional to Resolution Proof Size for 3-SAT

Entry ID: 1661f91ede76 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 22:27:03 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Resolution Proof Size for 3-SAT Instances

Statement:

For every 3-SAT instance Φ with n variables, the rank of its cycle matroid M_Φ satisfies $\text{rank}(M_\Phi) \leq \log_2(\rho(\Phi)) + 1$, where $\rho(\Phi)$ is the minimal resolution proof size of Φ . Equality holds for all instances with a unique minimal proof tree.

Rationale (proposer's reasoning):

Cycle matroid rank captures the interdependency structure of clauses, which directly influences resolution proof complexity. By mapping clause hyperedges to matroid circuits, we expose geometric constraints that correlate with proof length.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a36ff19f965e0b06

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = Fraction(A[i][i])
        for j in range(i, n):
            A[i][j] /= factor

        for k in range(i+1, n):
            factor = Fraction(A[k][i])
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]

    # Back-substitute to get the rank
    rank = 0
    for i in range(n):
        if all(abs(A[i][j]) < 1e-9 for j in range(n)):
            rank += 1
    return rank

def dpll_search(clauses, assignment):
    if not clauses:
        return True
    clause = next((c for c in clauses if any(l not in assignment or assignment[l] == False for l in c)))
    if not clause:
        return True

    for literal in clause:
        new_assignment = assignment.copy()
```

```

        new_assignment[literal] = True
        if dpll_search(clause, new_assignment):
            return True

        new_assignment[literal] = False
        if dpll_search(clause, new_assignment):
            return True

    return False

def resolution_proof_size(clauses):
    n = len(clauses)
    max_depth = 0
    for i in range(n):
        depth = 1
        while clauses[i]:
            new_clause = []
            for j in range(i+1, n):
                if any(l not in clauses[j] or clauses[j][l] == False for l in clauses[i]):
                    new_clause.extend(clauses[j])
            max_depth = max(max_depth, depth)
            clauses[i] = new_clause
            depth += 1
    return max_depth

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    num_clauses = random.randint(n, 2*n)
    variables = set(range(-n, 0)) | set(range(1, n+1))
    clauses = []
    for _ in range(num_clauses):
        clause = [random.choice(variables) for _ in range(3)]
        if len(set(clause)) == 3:
            clauses.append(clause)

    incidence_matrix = [[int(l in c or -l in c) for l in variables] for c in clauses]
    rank_M_phi = gaussian_elimination(incidence_matrix)

    rho_phi = resolution_proof_size(clauses)

    if rho_phi == 0:
        return {
            "metric_name": "rank(M_Φ)",
            "metric_value": rank_M_phi,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "resolution_proof_size_zero"
        }

    conjecture_holds = rank_M_phi <= math.log2(rho_phi) + 1
    counterexample = "" if conjecture_holds else f"rho(Φ)={rho_phi}, rank(M_Φ)={rank_M_phi}"

    return {
        "metric_name": "rank(M_Φ)",

```

```

        "metric_value": rank_M_phi,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 10**9) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    total_metric = sum(r["metric_value"] for r in results)
    mean_metric = total_metric / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample_desc = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c98e8243.py", line 122, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c98e8243.py", line 87, in run_trial
    clause = [random.choice(variables) for _ in range(3)]
               ~~~~~^
  File "/usr/lib/python3.12/random.py", line 348, in choice
    return seq[self._randbelow(len(seq))]
           ~~~~^
TypeError: 'set' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to `TypeError`, preventing data collection. Cannot assess conjecture validity without successful runs. | next: Fix the test code to handle variable selection properly (e.g., convert set to list) and rerun with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107031
2	preregistration	ollama_remote	qwen3:8b	0	0	24581
3	novelty	ollama_remote	qwen3:8b	0	0	20775
4	novelty	ollama_remote	qwen3:8b	0	0	12842
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13334
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13546
7	judge	ollama_remote	qwen3:8b	0	0	17912

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 210020 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/1661f91ede76.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1661f91ede76.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1661f91ede76.tar.gz` (if generated)